



Microcontroller & Interfacing

CE205T

	CLO-2		CLO-3					Total
Part	B	C	A	D	E	F	G	
Marks	50	50	50	50	50	50	50	
Obt.								

Project Name

RFID BASED ATTENDANCE SYSTEM

Group Number

1: ZAIN RASHID (BSCE-23041)

2: HASEEB ASHFAQ (BSCE-23048)

3: ABDUL JABBAR IJAZ (BSCE-23038)

A. Overview [CLO-3, 50 Marks]

This project implements a **RFID-based attendance system**. The goal is to automate and digitize attendance recording using RFID technology, improving speed, accuracy, and security compared to manual systems. When a user taps an RFID tag/card on the RFID reader (e.g., RC522), the STM32 reads the unique ID and logs the entry. This system can be used in schools, offices, or secured facilities for easy tracking and record-keeping.

- **Automate attendance tracking using RFID technology for accuracy and efficiency.**
- **Uniquely identify individuals through RFID cards or tags.**
- **Provide immediate feedback using LEDs and buzzers on successful or failed scans.**
- **Log or transmit attendance data for monitoring and record-keeping purposes.**

B. List of Components Used [CLO-2, 50 Marks]

List of the required components is given under.

Components	Cost	Datasheets	Operation
STM32F407G	15000 Rs	STM_DATASHEET	Controller
MFRC522 RFID card reader	550 Rs	RFID	Card readers
Oled Display	450 Rs	oled	Display
Bread Boards	300 Rs	No sheet	Bread boards
RFID cards/ RFID tags	90 Rs	No sheet	RFID cards
Wires	200 Rs	No sheet	Jumper wires
Ky006-Buzzer	80 RS	ky006	Buzzer

C. Peripherals of STM Microcontroller being used [CLO-2, 50 Marks]

1. GPIO (General Purpose Input/Output):

Port B: Used to control output signals, such as turning on the Red or Green LED to indicate valid or invalid RFID card scans.

Port A: Configured in Alternate Function Mode to interface with SPI1 for communication with the RFID module.

2. SPI (Serial Peripheral Interface):

SPI1: Configured for full-duplex communication, used to interface with the RC522 RFID Reader. Receives UID data from RFID tags and cards for authentication.

3. NVIC (Nested Vectored Interrupt Controller):

SPI1 Interrupt: Enabled and prioritized to handle incoming data from the RFID reader. An interrupt service routine (ISR) processes the received UID and verifies it against allowed entries.

4. SysTick Timer:

System Timekeeping: Generates millisecond delays and system ticks used in various parts of the program, such as debouncing and user feedback timing.

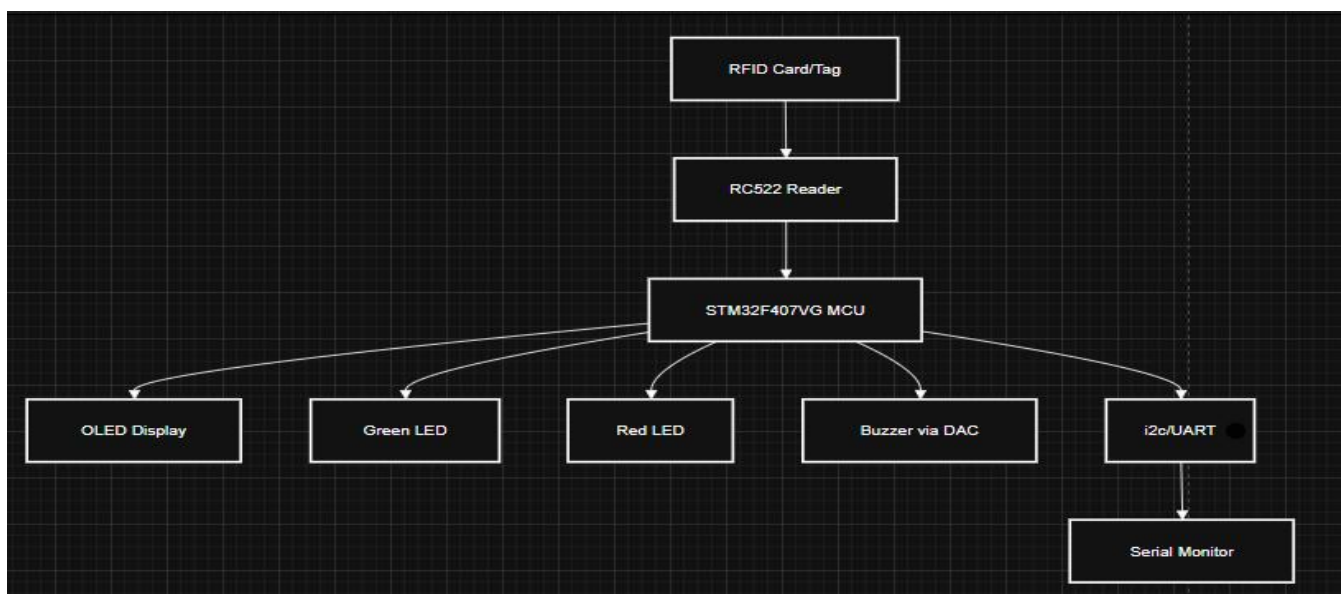
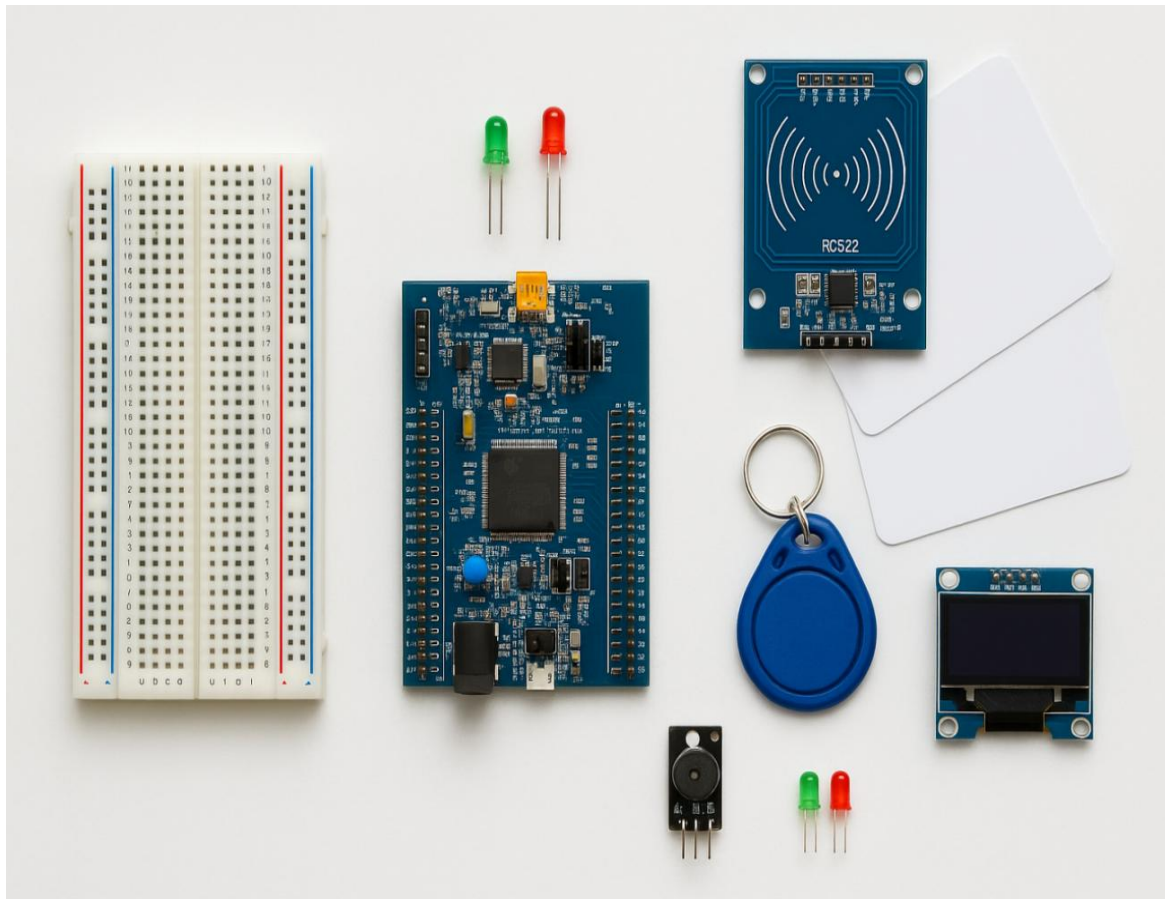
5. OLED Display (I2C/SPI Based):

User Feedback Interface: Displays messages like "Welcome," "Scan Card," "Access Granted," or "Access Denied." Acts as a simple and compact interface for the user to see system status.

6. HAL Library (Hardware Abstraction Layer):

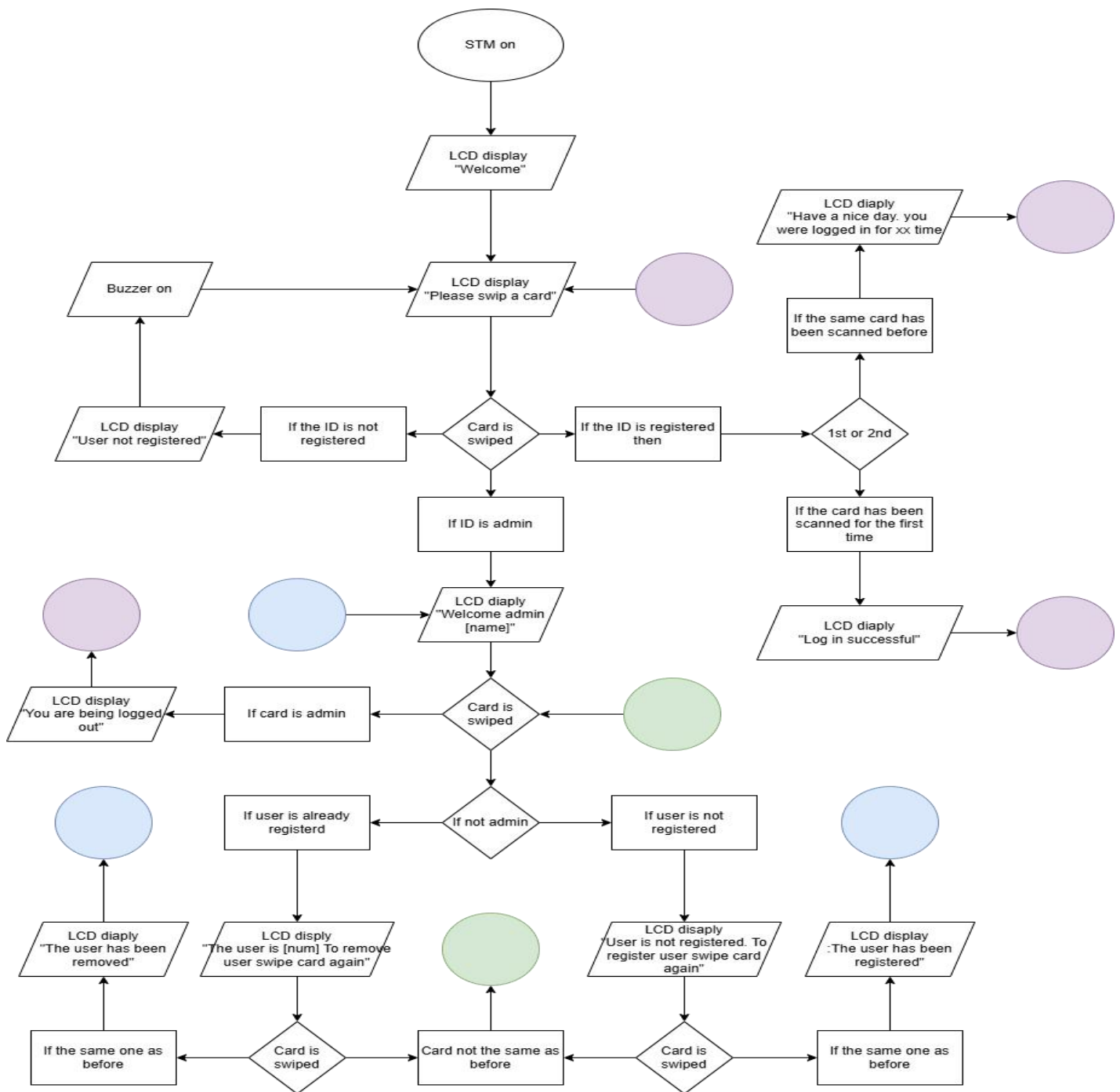
Peripheral Initialization: Provides a standardized and simplified way to initialize and configure GPIOs, SPI, SysTick, and OLED.

D. Block Diagram/Schematic [CLO-3, 50 Marks]



E. Flow Chart (Required at the time of final submission)

[CLO-3, 50 Marks]



F. CEP (Project Complexity) Attributes - Describe Briefly

[CLO-3, 50 Marks]

Attribute	Description	Complexity Level in your project
WP1: Depth of knowledge	The project shall involve in-depth engineering knowledge related to the area of Microprocessors, Microcontrollers & Interfacing [WK-4, Engineering Specialization].	The course provided comprehensive knowledge of microcontroller basics, embedded C programming, peripheral configuration, and real-time operating systems, which were crucial for the project. Practical labs, workshops, and project management sessions facilitated the hands-on experience needed to successfully develop the system using the STM32F407 Discovery board.
WP2: Range of conflicting requirements	The project has multiple conflicting requirements in terms of optimal usage of peripheral resources available on a Microcontroller.	Conflicts include MFRC522 header files issue. Some interfacing with 12c and limited GPIO pins for multiple peripherals and managing multiple interrupt sources with restricted interrupt lines. Additionally, peripheral multiplexing and power consumption pose significant challenges..
WP5 Extent of applicable codes	The projects expose the students to broadly defined problems which require the development of codes that may be partially outside those encompassed by well-documented standards.	The code complexity involved integrating multiple peripherals, some issues on stm32 cube ide while interfacing with i2c and handling real-time tasks with interrupt-driven programming for user interaction, card validation, and display updates..
WP7 Interdependence	The projects shall have multiple components at the hardware and software level.	The main challenge encountered while interfacing with the components was effectively managing interrupt-driven communication between the STM32 microcontroller and external devices (e.g., RFID reader) while ensuring seamless operation of the OLED display and GPIO-based indicators, all within the constraints of limited GPIO pins and interrupt lines.

G. Code [CLO-3, 50 Marks]

```
#include "main.h"
#include "rc522.h"
#include "ssd1306.h"
#include "ssd1306_fonts.h"
#include <string.h>
#include <stdio.h>

SPI_HandleTypeDef hspi1;
I2C_HandleTypeDef hi2c1;

uint8_t status;
uint8_t str[MAX_LEN];
uint8_t sNum[5];

uint8_t valid_uids[][4] = {
    {185, 76, 62, 2},
    {115, 203, 30, 45}
};

int is_valid_card(uint8_t *uid) {
    for (int i = 0; i < sizeof(valid_uids) / sizeof(valid_uids[0]); i++) {
        if (memcmp(uid, valid_uids[i], 4) == 0)
            return 1;
    }
    return 0;
}

void display_message(const char* msg) {
    ssd1306_Fill(Black);
    ssd1306_SetCursor(0, 0);
    ssd1306_WriteString(msg, Font_7x10, White);
    ssd1306_UpdateScreen();
}

void blink_invalid_card_led() {
    for (int i = 0; i < 3; i++) {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
        HAL_Delay(200);
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    }
}
```

```

        HAL_Delay(200);
    }
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET); // Buzzer ON
    HAL_Delay(3000);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET); // Buzzer OFF
}

```

```

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
static void MX_I2C1_Init(void);

```

```

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_I2C1_Init();

    MFRC522_Init();
    ssd1306_Init();

    display_message("Scan RFID Card");

    while (1) {
        status = MFRC522_Request(PICC_REQIDL, str);
        if (status == MI_OK) {
            status = MFRC522_Anticoll(str);
            if (status == MI_OK) {
                memcpy(sNum, str, 4);
                if (is_valid_card(sNum)) {
                    display_message("Access Granted");
                } else {
                    display_message("Access Denied");
                    blink_invalid_card_led();
                }
            }
            HAL_Delay(2000);
            display_message("Scan RFID Card");
        }
    }
}

```



```
static void MX_I2C1_Init(void) {
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK) {
        Error_Handler();
    }
}
```

H. References

- <https://thongrobot.wordpress.com/2017/07/22/stm32f407-rfid-project- full-code/>
- <https://stm32f4-discovery.net/2014/07/library-23-read-rfid-tag-mfrc522- stm32f4xx-devices/>
- <https://community.st.com/t5/stm32-mcus-boards-and-hardware/using- rc522-module-on-stm32f401/td-p/659916>
- <https://blog.embeddedexpert.io/?p=768>

<https://circuitdigest.com/microcontroller-projects/interface-rfid-with-stm32-microcontroller>